



VISUAL PROGRAMMING

Animation

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

OUTLINES

- Introduction
- Path Transition
- Fade Transition
- Timeline

INTRODUCTION

- In general, animating an object implies creating illusion of its motion by rapid display.
- In JavaFX, a node can be animated by changing its property over time.
- JavaFX provides a package named `javafx.animation`.
- JavaFX provides the `Animation` class with the core functionality for all animations.

ANIMATION CLASS

javafx.animation.Animation

-autoReverse: BooleanProperty
-cycleCount: IntegerProperty
-rate: DoubleProperty
-status: ReadOnlyObjectProperty
 <Animation.Status>

+pause(): void
+play(): void
+stop(): void

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

Defines whether the animation reverses direction on alternating cycles.
Defines the number of cycles in this animation.
Defines the speed and direction for this animation.
Read-only property to indicate the status of the animation.

Pauses the animation.
Plays the animation from the current position.
Stops the animation and resets the animation.

ANIMATION

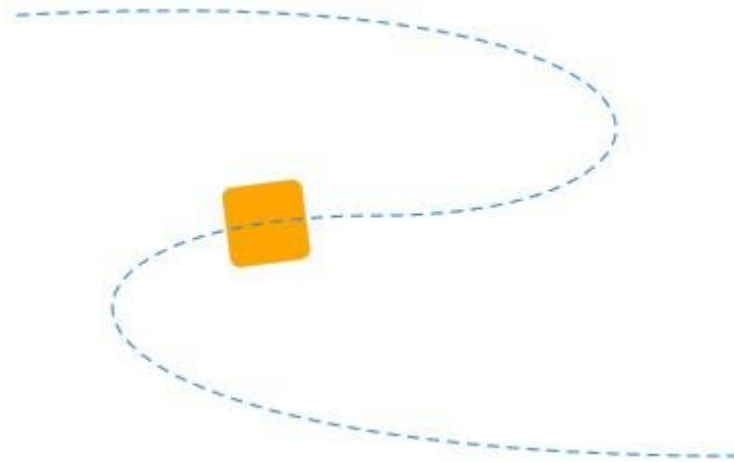
- **Animation in JavaFX can be divided into timeline animation and transitions.**
- **Timeline and Transition are subclasses of the `javafx.animation.Animation` class.**

TRANSITION EXAMPLES

- **Fade Transition**
- **Fill Transition**
- **Rotate Transition**
- **Scale Transition**
- **Stroke Transition**
- **Translate Transition**
- **Path Transition**
- **Sequential Transition**
- **Parallel Transition**

PATH TRANSITION

- A path transition moves a node along a path from one end to the other over a given time.



PATH TRANSITION

javafx.animation.PathTransition

-duration: ObjectProperty<Duration>
-node: ObjectProperty<Node>
-orientation: ObjectProperty
 <PathTransition.OrientationType>
-path: ObjectType<Shape>

+PathTransition()
+PathTransition(duration: Duration,
 path: Shape)
+PathTransition(duration: Duration,
 path: Shape, node: Node)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The duration of this transition.

The target node of this transition.

The orientation of the node along the path.

The shape whose outline is used as a path to animate the node move.

Creates an empty PathTransition.

Creates a PathTransition with the specified duration and path.

Creates a PathTransition with the specified duration, path, and node.

PATH TRANSITION EXAMPLE

PathTransitionDemo

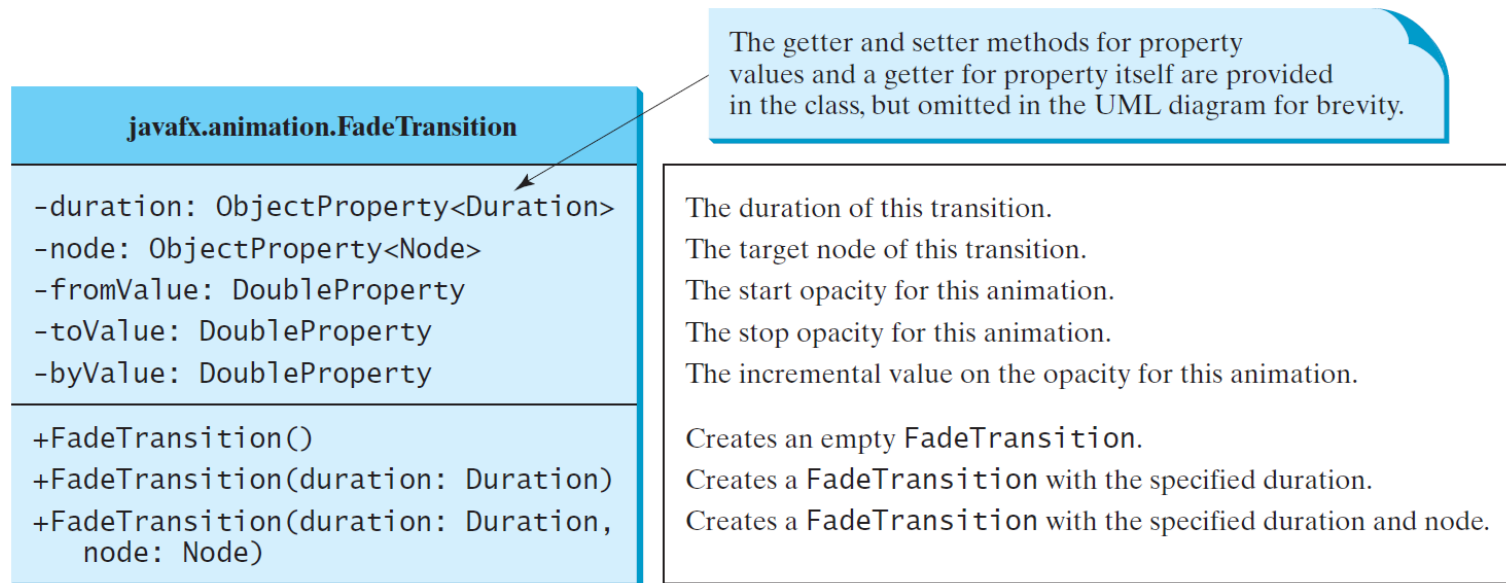
```
// Create a path transition
PathTransition pt = new PathTransition();
pt.setDuration(Duration.millis(4000));
pt.setPath(circle);
pt.setNode(rectangle);
pt.setOrientation(PathTransition.OrientationType.ORTHOGONAL_TO_TANGENT);
pt.setCycleCount(Timeline.INDEFINITE);
pt.setAutoReverse(true);
pt.play(); // Start animation
```

PATH TRANSITION

- **Duration:**
 - The duration of this Transition in milliseconds.
- **Node:**
 - The target node of this PathTransition.
- **Orientation:**
 - Specifies the upright orientation of node along the path.
 - Orientation has two possible values:
 - **NONE**
 - The targeted node's rotation matrix stays unchange along the geometric path.
 - **ORTHOGONAL_TO_TANGENT**
 - The targeted node's rotation matrix is set to keep node perpendicular to the path's tangent along the geometric path.
- **Path:**
 - The shape on which outline the node should be animated.

FADE TRANSITION

- The FadeTransition class animates the change of the opacity in a node over a given time.



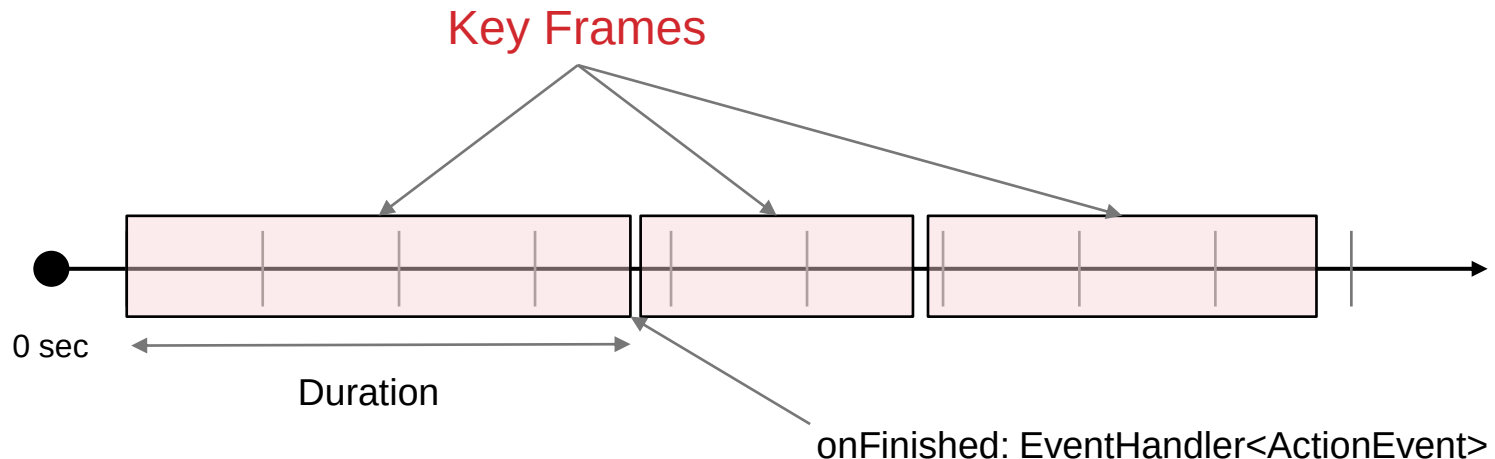
FADE TRANSITION EXAMPLE

FadeTransitionDemo

```
// Apply a fade transition to ellipse  
FadeTransition ft = new FadeTransition(Duration.millis(3000), ellipse);  
ft.setFromValue(1.0);  
ft.setToValue(0.1);  
ft.setCycleCount(Timeline.INDEFINITE);  
ft.setAutoReverse(true);  
ft.play(); // Start animation
```

TIMELINE

- **The Timeline class can be used to program any animation using one or more KeyFrames.**
- **Each KeyFrame is executed sequentially at a specified time interval.**
- **Timeline inherits from Animation.**



TIMELINE

TimelineDemo

```
// Create a handler for changing text
EventHandler<ActionEvent> eventHandler = e -> {
    if (text.getText().length() != 0) {
        text.setText("");
    } else {
        text.setText("Programming is fun");
    }
};

// Create an animation for alternating text
Timeline animation = new Timeline(new KeyFrame(Duration.millis(500), eventHandler));
animation.setCycleCount(Timeline.INDEFINITE);
animation.play(); // Start animation
```

ANY QUESTIONS?